@codedsoul_05

IRCTC SD HANDBOOK
v1.0 ★

IRCTC SYSTEM DESIGN

– HANDWRITTEN HANDBOOK –

Designing India's Largest Online Railway Reservation System ☺

IRCTC



"Right Train.
Right Seat.
Right Time."

High Availability • Scalability • Reliability • Performance • Security

What's Inside ?

1. System Overview & Requirements
2. High Level Architecture
3. Core Services & Modules
4. Seat Allocation & Inventory
5. Database Design
6. Payment Flow
7. Scaling & Performance
8. Security, Bots & Fair Usage
9. Failure Handling & Recovery
10. End-to-End Flow & Best Practices

+ Cheat Sheet & Interview Q&A

IRCTC by the Numbers (Approx)

Daily Users	10 – 15 Million+
Daily Bookings	~ 10 – 12 Lakh
Peak Requests	~ 1 – 2 Lakh RPS
Tatkal Time	10:00 AM (Rush Hour!)
Databases	Sharded + Read Replicas
Availability Target	99.95%

Topics Covered

- ✓ System Design
- ✓ Databases
- ✓ Caching (Redis)
- ✓ Queues (Kafka)
- ✓ APIs & Services
- ✓ Scalability
- ✓ High Availability
- ✓ Payment Systems
- ✓ Security & Bots
- ✓ Monitoring & Alerts
- ✓ Failure Handling
- ✓ Best Practices

Who is this for?

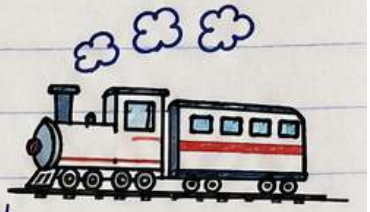
- ☆ System Design Interview Preparation
- ☆ Backend / Full Stack Developers
- ☆ Anyone who loves building scalable systems!

☺

"A great system is not built by chance, it is designed with clarity, tested with failure, and scaled with care."

Think Like a System Designer. Build for Scale. Ship with Confidence. ★

IRCTC SYSTEM DESIGN



Let's Design the system that powers Indian Railways ticket booking for millions of users ☺

1 Problem Statement

Design a highly scalable system that allows users to search trains, check availability, book tickets and make payments – especially during peak load like Tatkal Booking.

Why it's Hard ?

- Huge traffic in short time
- Limited seats
- Prevent fraud & bots
- Ensure fairness
- High availability

2 Daily Traffic (Estimates)

- 2+ Million users on normal days
- 10+ Million users on peak days
- Tatkal opening (10 AM) – up to 2 Million users in few minutes
- 20K – 50K bookings per second during peak
- Read heavy system (Search, Availability) >> Write heavy system (Booking)

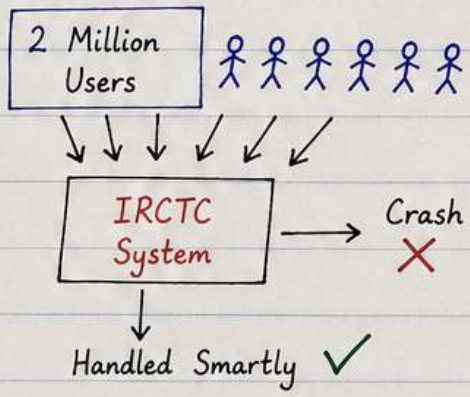
Peak Time

Tatkal : 10 AM

General : Anytime

3 Tatkal Challenge

- All tatkal quota seats open at 10:00 AM
- Users flood the system at the same second
- Need to handle massive concurrent requests
- Ensure fair distribution of seats
- Avoid system crash



4 Functional Requirements

1. User Registration & Login
2. Search Trains / Check Availability
3. Select Train, Class, Quota
4. Book Ticket
5. Make Payment
6. Get Confirmation (PNR)
7. Cancel Ticket
8. Refund (if applicable)
9. Notifications (SMS / Email)

5 Non Functional Requirements

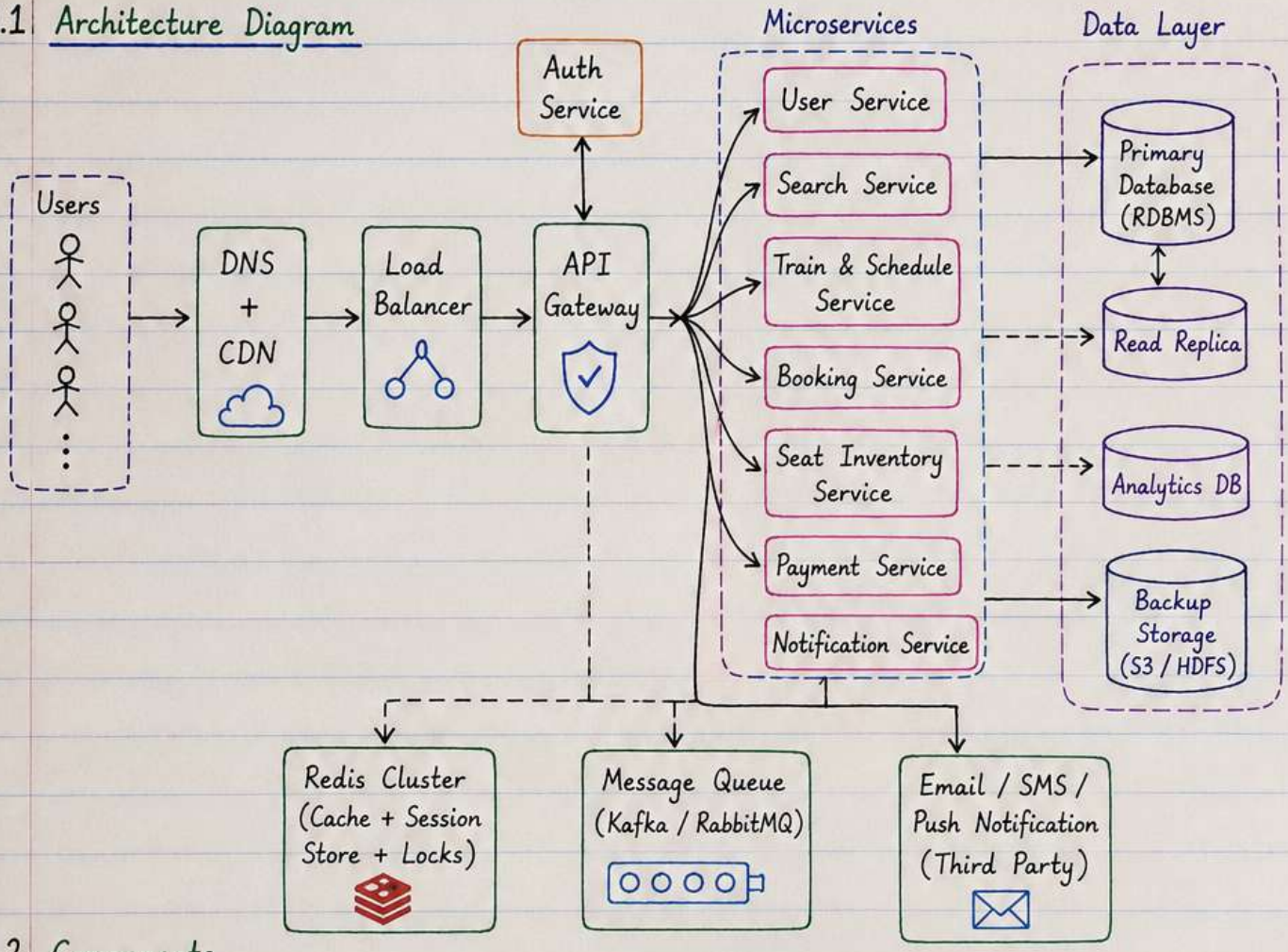
- High Availability → 99.99%
- Scalability → Handle Millions of users
- Low Latency → Fast Response
- Data Consistency → No double booking
- Security → Prevent fraud
- Fault Tolerance → System should not go down
- Monitoring & Alerting → Real time alerts

Goal : Build a system that is Fast, Fair, Scalable and Reliable even under extreme load.

2. HIGH LEVEL ARCHITECTURE

We will design a highly available, scalable and fault tolerant system that can handle millions of concurrent users. ☺

2.1 Architecture Diagram



2.2 Components

1. Users → Web, Mobile App, Agents
2. DNS + CDN → Route users to nearest edge server, static content caching
3. Load Balancer → Distribute traffic across multiple servers
4. API Gateway → Single entry point, request routing, rate limiting
5. Auth Service → Handles login, signup, token validation
6. Microservices → Each service handles a specific business capability
7. Data Layer → Stores all persistent data, replicas for read scalability
8. Redis Cluster → Caching, session store, distributed locks, counters
9. Message Queue → Async processing of emails, notifications, refunds
10. Notification Service → Send email, sms, push notifications

Why Microservices?

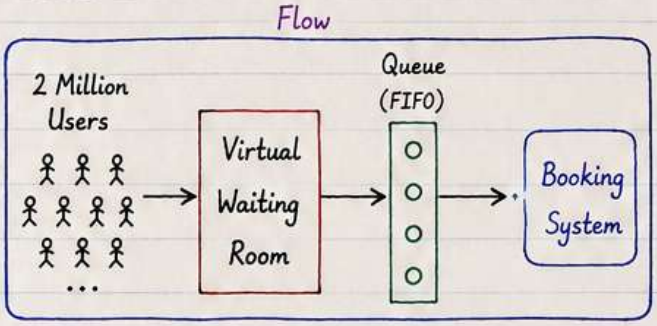
- Independent scaling of services
- Fault isolation → failure in one service won't affect others
- Easy to deploy and update
- Better maintainability

3. HOW 2 MILLION USERS JOIN AT 10 AM ?
(Handling Traffic Spike)

At 10:00 AM (Tatkal opening), ~2 Million users try to access the system at the same time. We need to ensure – system doesn't crash and fair access for everyone. ☺

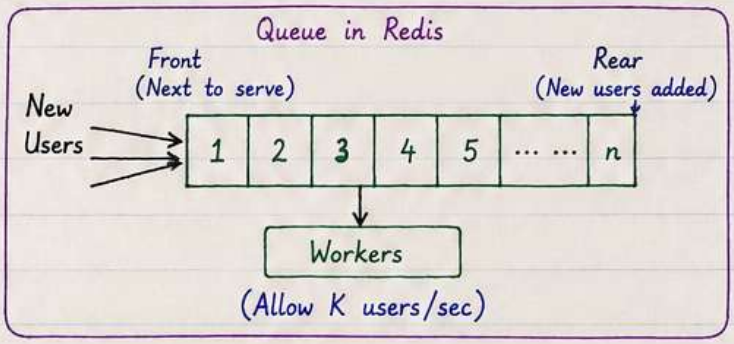
3.1 Virtual Waiting Room

- All users land in a Virtual Waiting Room instead of directly hitting the booking system.
- Show Position / Status / Estimated time.
- Users are placed in a Queue.



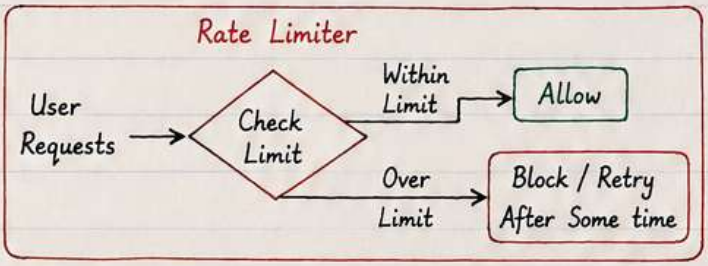
3.2 Queue Mechanism

- Use FIFO Queue for fairness.
- Each user gets a unique token / position.
- Token is stored in Redis (in-memory) for speed.
- Only a limited number of users are allowed to proceed to booking system at a time.



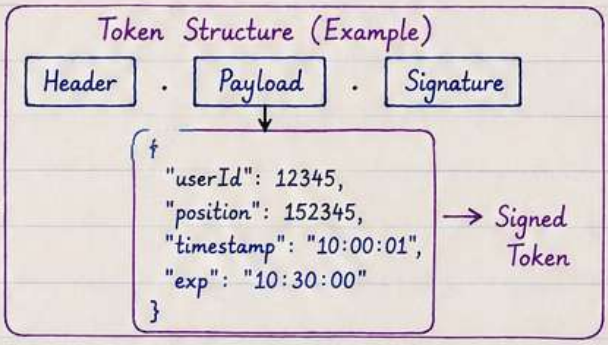
3.3 Rate Limiting

- Each IP / Device / Account has a rate limit.
- Prevents a single user or bot from sending too many requests.
- Example: Max 10 requests / second / IP



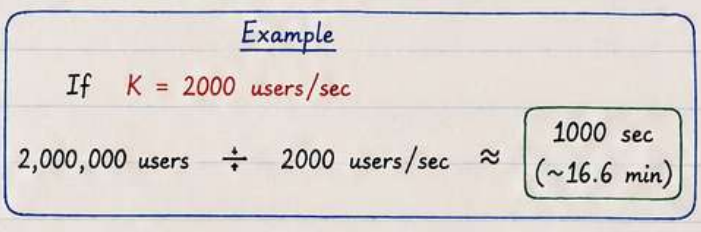
3.4 Token Generation

- When user enters queue, system generates a secure token (JWT / Random Token).
- Token contains → userId, position, timestamp, signature
- Token is used to check the user's position and access booking system when it's their turn.



3.5 Allow Users in Batches

- System lets only K users per second to proceed.
- K is decided based on system capacity.
- This keeps the backend safe from overload.



Key Points

- Queue + Rate Limiting + Token helps to handle massive traffic.
- Fairness maintained, system remains stable.
- Users see waiting time, don't feel the system is down. ☺

4. SEAT ALLOCATION & INVENTORY MANAGEMENT

Goal : Ensure no two users get the same seat and handle concurrent requests safely. ☺

4.1 Seat Inventory Model

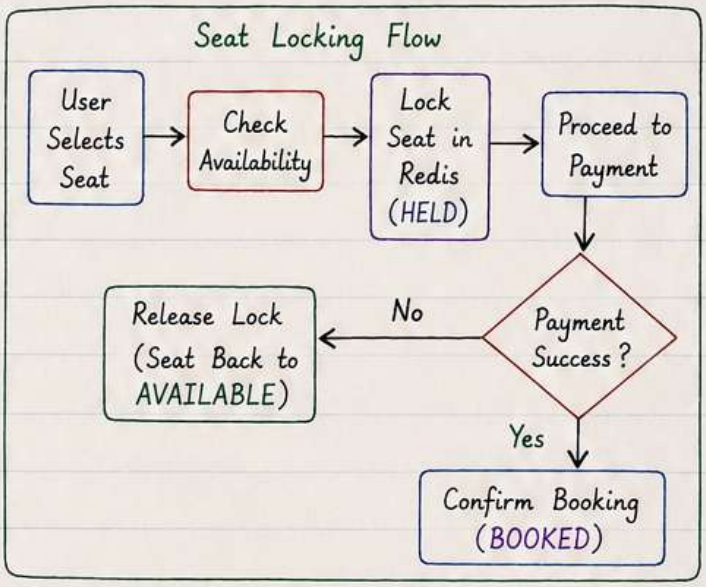
- Every train has a fixed quota of seats per class & quota type.
- Inventory is updated in real-time.
- Different quotas : General, Tatkal, Ladies, Senior Citizen, etc.
- Seats go through multiple states.

Seat States

State	Description
AVAILABLE	Seat is free and can be booked
HELD	Temporarily locked for a user during booking flow
BOOKED	Payment successful, ticket confirmed
CANCELLED	Booking cancelled, seat released
EXPIRED	Hold expired, seat released back

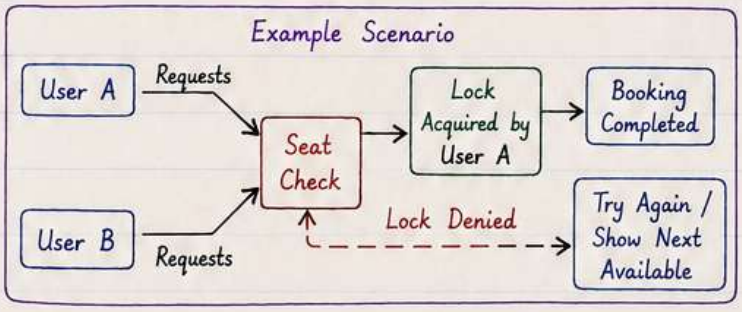
4.2 Seat Locking (Prevent Double Booking)

- When a user selects seat(s), we lock the seats for a short period (e.g., 10 minutes).
- During this time, only that user can continue the booking.
- If payment not completed within timeout → lock expires and seat becomes AVAILABLE.
- Implemented using Redis Distributed Locks.



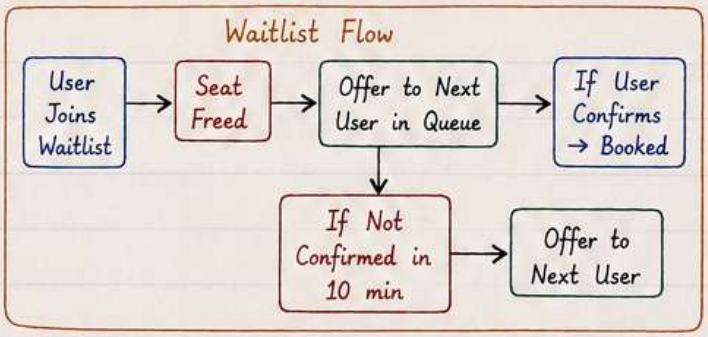
4.3 Handling Concurrency (Race Conditions)

- Multiple users may try to book the same seat at the same time.
- Use DB transactions + row level locking (SELECT ... FOR UPDATE).
- Or use Redis distributed locks before writing to database.
- This ensures only one user can book the seat.



4.4 Waiting List Management

- If seats are not available, user is added to waitlist.
- Waitlist is per train, class, quota.
- When a seat gets freed (cancellation / expiry), system offers to next user in waitlist.
- Users get a limited time (e.g., 10 min) to complete booking, else offer goes to next.



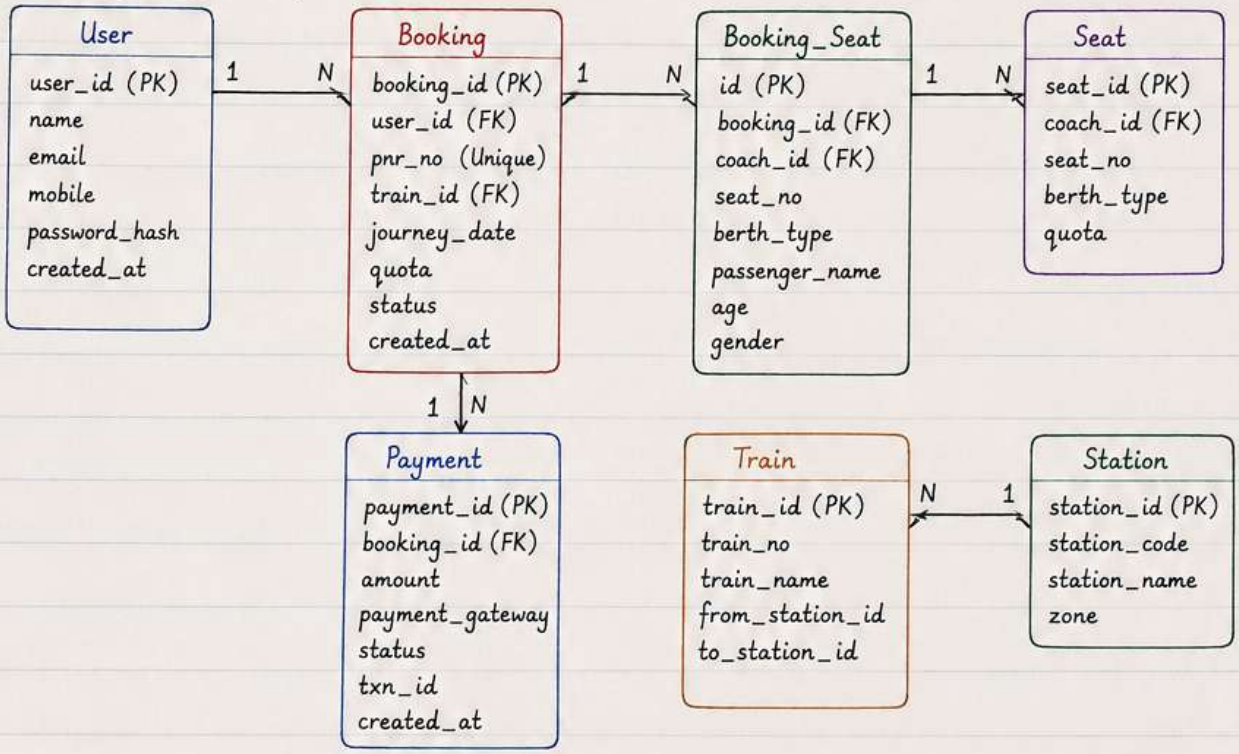
Key Points

- ✓ Short term locks prevent double booking.
- ✓ Locks must have TTL (expiry) to release seats.
- ✓ Handle concurrency at DB + Cache level.
- ✓ Waitlist helps in better seat utilization.

5. DATABASE DESIGN

We need a well structured schema that supports high read + write, concurrency and consistency. ☺

5.1 ER Diagram (Simplified)



5.2 Table Details

- **User**
  - Stores user account information.
  - Key Indexes :
    - user_id (PK)
    - mobile (Unique)
- **Train**
  - Stores train master data.
  - Key Indexes :
    - train_id (PK)
    - train_no (Unique)
- **Seat**
  - Stores seat layout for each coach.
  - Key Indexes :
    - seat_id (PK)
    - (coach_id, seat_no) → Unique
- **Booking**
  - Stores booking header information.
  - Key Indexes :
    - booking_id (PK)
    - pnr_no (Unique)
    - user_id
    - train_id, journey_date
- **Booking_Seat**
  - Stores passenger/seat details.
  - Key Indexes :
    - id (PK)
    - booking_id
    - (coach_id, seat_no)
- **Payment**
  - Stores payment transactions.
  - Key Indexes :
    - payment_id (PK)
    - booking_id
    - txn_id (Unique)
- **Station**
  - Stores station master data.
  - Key Indexes :
    - station_id (PK)
    - station_code (Unique)

5.3 Additional Notes

- Use proper indexing to speed up search & availability.
- Data is normalized to avoid redundancy.
- Consider read replicas for read heavy tables like Seat, Train, Station.
- Old bookings can be archived to cold storage.

Important Indexes (Summary)

- ☆ Booking : (user_id), (train_id, journey_date), pnr_no
- ☆ Booking_Seat : (booking_id), (coach_id, seat_no)
- ☆ Seat : (coach_id, seat_no), (quota)
- ☆ Payment : (booking_id), (status)
- ☆ Train : (train_no)
- ☆ Station : (station_code)

Why this design?

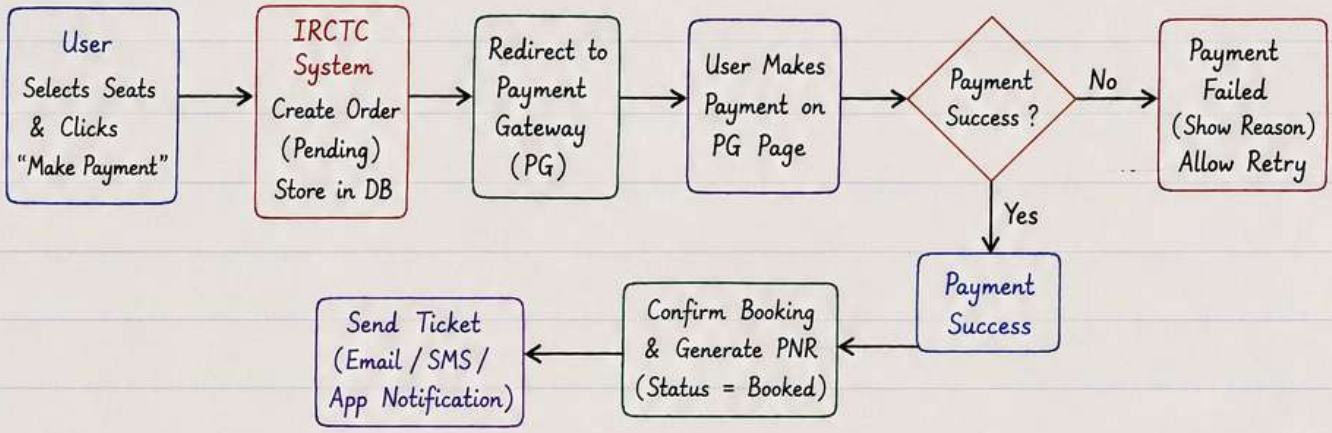
- ✓ Supports high concurrency
- ✓ Scalable & maintainable
- ✓ Ensures data integrity
- ✓ Fast reads and writes

Good DB design = 50% of the system success! ☺

6. PAYMENT FLOW

Goal : Ensure secure, reliable and idempotent payment processing. ☺

6.1 Payment Process Flow



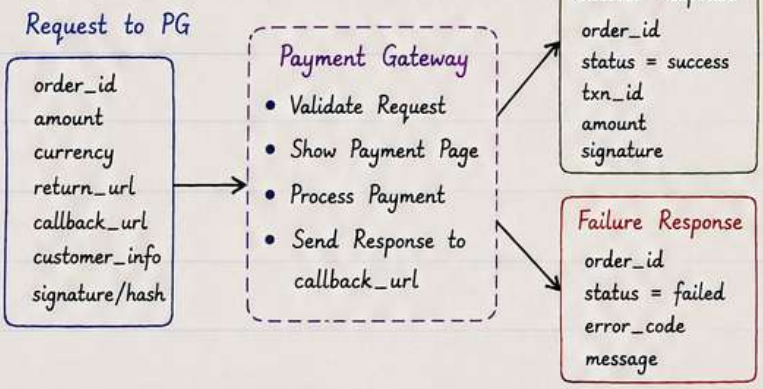
6.2 Payment States

State	Description	Next State
INITIATED	Order created, user redirected to PG	PROCESSING
PROCESSING	Payment in progress at PG	SUCCESS or FAILED
SUCCESS	Payment successful, booking confirmed	COMPLETED
FAILED	Payment failed / cancelled / timeout	ALLOW RETRY
REFUNDED	Refund issued (if applicable)	COMPLETED

6.3 Important Points

- Every payment request has a unique order_id.
- Order + Payment updates are done in atomic transaction.
- Use idempotency key to avoid duplicate payments (same order_id).
- If payment success but system didn't receive response → verify using order_id.
- Store payment response from PG for audit.
- Never store card details (use tokenization).

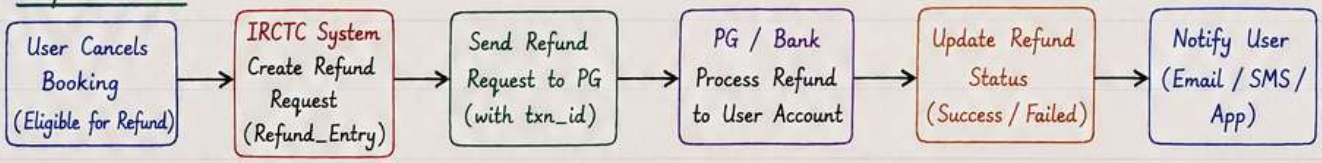
6.4 Payment Gateway Interaction



6.5 Failure & Retry Handling

- If payment fails → allow user to retry.
- If payment pending for long → show "Payment Pending" and auto verify in background.
- If user closes page → we show status from our system (order_id based).
- If payment success but booking failed → auto refund (or manual review).
- Refunds are processed via PG / Bank.

6.6 Refund Flow



Key Takeaways

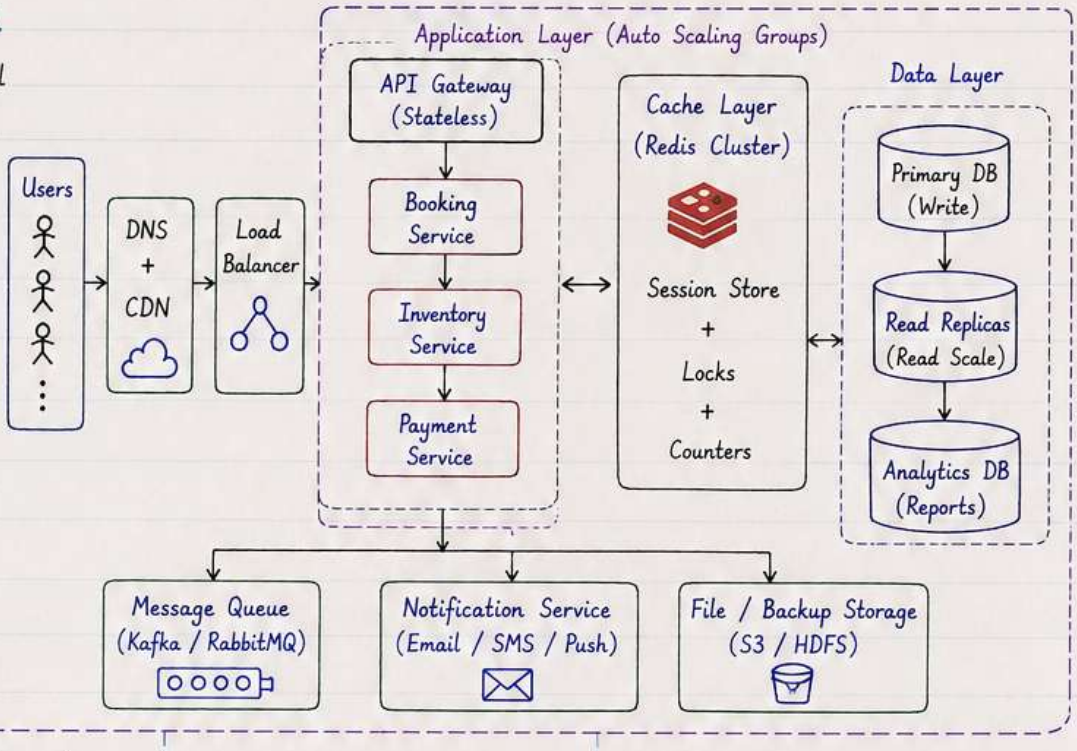
- Always use order_id + idempotency key.
- Handle all edge cases (pending / failed / timeout).
- Payments & Bookings must be atomic.
- Secure communication with PG (HTTPS + Sign).
- Logs & monitoring are very important.
- Better to fail safe than fail open! ☺

7. SCALING & PERFORMANCE

Goal : Handle huge traffic, provide low latency and high availability. ☺

7.1 Scaling Overview

- Traffic during Tatkal time is highly spiky.
- We need to scale both vertically and horizontally.
- Use caching, queues, and database scaling to handle load.



7.2 Caching Strategy (Redis)

- Cache frequently read data.
  - Train List
  - Station List
  - Seat Availability (short term)
  - User Session
- Use Redis for ultra fast read/write.
- Set TTL for keys to keep data fresh.

7.3 Database Scaling

- Use Read Replicas for read heavy queries.
- Shard data by train_id, date or station for large tables.
- Partition booking data by date (e.g., monthly partitions).
- Use connection pooling and proper indexing.

7.4 Horizontal Scaling

- All services are stateless.
- Deploy multiple instances behind Load Balancer.
- Auto scale based on CPU, Memory, Request Rate.
- Use containers (Docker) and orchestration (Kubernetes).

7.5 CDN (Content Delivery Network)

- Serve static content (images, CSS, JS, fonts) from CDN.
- Reduces load on origin servers and improves response time.
- Example : CloudFront, Cloudflare, Fastly.

7.6 Other Optimization

- Compress responses (Gzip / Brotli).
- Use HTTP/2 or HTTP/3.
- Keep-alive connections.
- Optimize queries and avoid N+1 problems.
- Monitor and profile slow endpoints.

Key Scaling Components

- ✓ Stateless Services
- ✓ Auto Scaling Groups
- ✓ Caching (Redis)
- ✓ Queue for async processing
- ✓ Read Replicas
- ✓ Database Sharding
- ✓ CDN for static content
- ✓ Connection Pooling
- ✓ Monitoring & Alerting
- ✓ Load Testing
- ✓ Failover & Multi-AZ Deployment

☺

Traffic Handling Example



Normal Day Traffic	:	~ 100K users/day
During Tatkal Time	:	~ 2 Million users in few minutes
System Capacity	:	Scale to 10x during spike
After Peak	:	Scale down to normal

Auto Scaling helps us keep cost low and performance high. ☺

Monitoring Stack

- Prometheus
- Grafana
- ELK Stack (Logs)
- Alertmanager

8. PREVENT BOTS, AGENTS & UNFAIR USAGE

Goal : Ensure fair access for real users and prevent abuse of the system. ☺

8.1 Threats

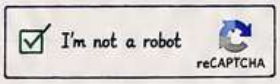
- Bots / Scripts booking in milliseconds.
- Multiple accounts from same device / IP.
- Ticket agents using automation & proxies.
- Hoarding of confirmed tickets and reselling.

Attack Flow (Example)



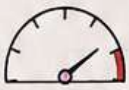
8.2 Counter Measures

CAPTCHA / Human Verification



- Add CAPTCHA during login, signup and high risk actions.
- Use Google reCAPTCHA v3/v2.

Rate Limiting



- Limit requests per IP / Device / User.
- Different limits for login, search, availability check, booking.
- Redis based rate limiter.

Device Fingerprinting



- Identify unique device using fingerprint (browser, plugins, screen, fonts, etc).
- Detect suspicious device change.

Account Limits



- Limit number of tickets per user per day / per journey.
- Restrict on number of passengers per booking.

IP / Network Protection



- Block / challenge suspicious IP ranges.
- Use IP reputation services.
- Detect datacenter / proxy IPs.

Behavioral Analysis



- Monitor unusual patterns (too fast clicks, constant availability checks).
- ML / rules to detect bots.

8.3 Additional Techniques

- Enforce login + phone verification.
- Use 2-Factor Authentication.
- Cool-off period after failed booking.
- Detect automation using timing patterns (human vs bot).
- Hide API endpoints, use WAF.
- Use JS challenges / Proof of Work for high priority flows.

Rate Limit Example (Per IP)

Action	Limit	Window
Login	5 requests	1 minute
Search Trains	60 requests	1 minute
Check Availability	30 requests	1 minute
Book Ticket	3 requests	1 minute
Payment Initiate	5 requests	5 minutes

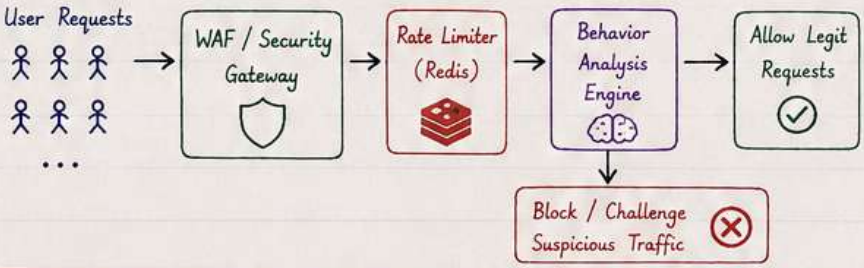
Why These Measures?

- ✓ Protect system from abuse
- ✓ Ensure fair chance for genuine users
- ✓ Improve system stability
- ✓ Reduce load & cost
- ✓ Build trust and reliability

☺

8.4 Monitoring & Alerting

- Monitor metrics in real-time.
- Alert on abnormal spikes.
- Use dashboards & logs.
- Auto block or throttle malicious traffic.



Key Takeaways

- ☆ Queue + Rate Limiting
- ☆ CAPTCHA + Device Fingerprinting
- ☆ Account & IP limits
- ☆ Behavioral Analysis
- ☆ Monitoring + Alerts
- ☆ Automation Detection
- ☆ Ensure fair access
- ☆ Happy real users ☺

9. FAILURE HANDLING & RECOVERY

Goal : System should be fault tolerant and recover automatically with minimal data loss. ☺

9.1 Types of Failures

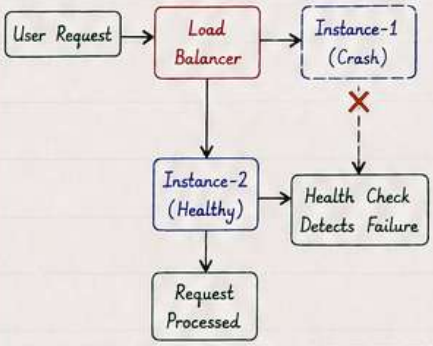
Failure Type	Examples
Server Failure	Booking service instance crashes.
Database Failure	DB down, slow, or primary node failure.
Network Failure	Packet loss, timeout, DNS issues.
Payment Failure	PG timeout, no response, partial success.
Queue Failure	Kafka/RabbitMQ broker down.
Cache Failure	Redis down or data loss.
Third Party Failure	SMS/Email/Bank server down.

9.2 General Strategies

- **Redundancy** : Multiple instances, replicas, AZ.
- **Timeouts** : Set proper timeouts for all calls.
- **Retries** : Retry with exponential backoff.
- **Circuit Breaker** : Stop calling failing service.
- **Bulkhead Isolation** : Failures in one service should not impact others.
- **Idempotency** : Duplicate requests should not create duplicate bookings/payments.
- **Monitoring & Alerts** : Detect issues early.
- **Graceful Degradation** : Show partial results instead of complete failure.

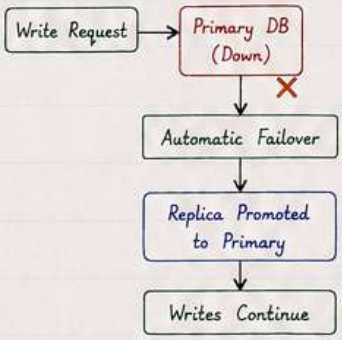
9.3 Failure Handling Scenarios

A. Server Crash (e.g., Booking Service)



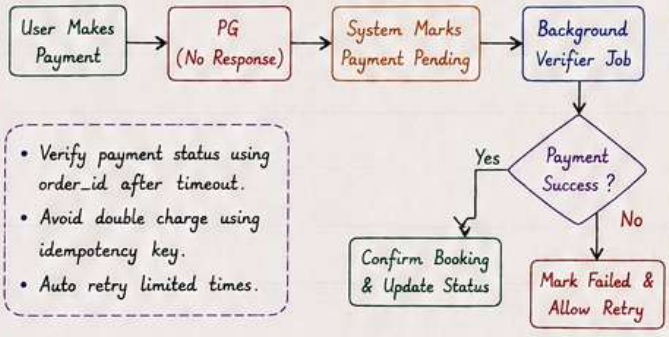
- Health check removes failed instance.
- Traffic routed to healthy instances.
- Auto scaling launches new instance if needed.

B. Database Failure (Primary Down)



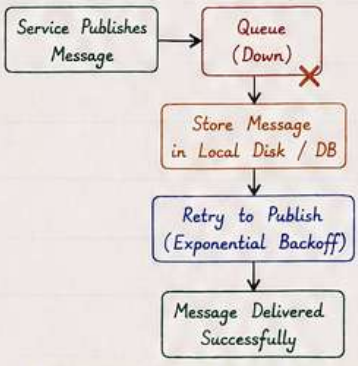
- Use Multi-AZ for high availability.
- Replication ensures no data loss.
- Regular backups & PITR for safety.

C. Payment Timeout / No Response



- Verify payment status using order_id after timeout.
- Avoid double charge using idempotency key.
- Auto retry limited times.

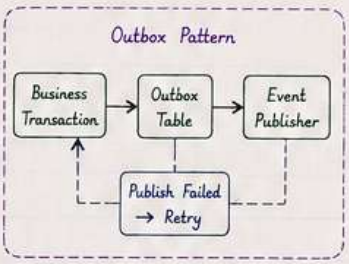
D. Queue / Message Broker Down



- Don't lose messages.
- Persist locally and retry.
- Use Dead Letter Queue (DLQ) for poison messages.

9.4 Data Consistency

- Use ACID transactions for critical operations.
- Use eventual consistency for non-critical data. (e.g., analytics, reports).
- Use outbox pattern for reliable event publishing.
- Regular reconciliation jobs to fix inconsistencies.



9.5 Monitoring & Recovery Tools

- Metrics : Prometheus / Grafana
- Logs : ELK Stack
- Tracing : Jaeger / Zipkin
- Alerts : PagerDuty / Opsgenie
- Backups : Daily Full + Continuous Logs
- DR Plan : RPO & RTO defined.





☺

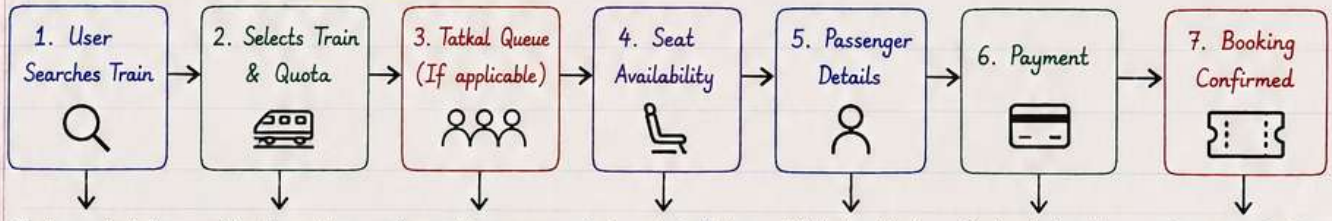
Key Takeaways

- ✓ Detect failures quickly.
- ✓ Fail gracefully, not completely.
- ✓ Auto recover wherever possible.
- ✓ Use retries, timeouts & circuit breaker.
- ✓ Always design for failure.
- ✓ Monitor, Test & Improve continuously.
- ✓ Better to fail safe than fail open.
- ✓ Users should get correct information always. ☺

10. END-TO-END IRCTC BOOKING FLOW REVISION SHEET & INTERVIEW CHEAT SHEET

Let's put everything together – from user opens app to ticket in hand! ☺

10.1 End-to-End Booking Flow (High Level)



- Trains, schedule, fares from DB / Cache
- Check quota & seat availability
- User enters queue, gets token, then allowed
- Lock seat in Redis, confirm availability atomically
- Validate details, apply rules (Age, ID, etc.)
- Redirect to PG, payment success, get response
- Generate PNR, update inventory, send ticket

☆ All critical steps are atomic, idempotent and logged for reliability.

10.2 Key Components Recap

- Users : Web / App / Agents
- Load Balancer : Distributes traffic
- Services : Booking, Inventory, Payment, Notification
- Cache (Redis) : Sessions, Locks, Counters, Rate limiting
- DB (MySQL) : Core data – bookings, trains, users
- Read Replicas : Handle read heavy queries
- Queue (Kafka) : Async jobs – emails, sms, notifications
- Storage (S3) : Tickets, backups, reports
- Monitoring : Prometheus, Grafana, ELK, Alerts

10.3 Important Design Principles

- ☑ Stateless services behind Load Balancer
- ☑ Horizontal scaling for all services
- ☑ Cache + Queue to handle spikes
- ☑ Atomic operations with idempotency
- ☑ Data partitioning & Read Replicas
- ☑ CDN for static content
- ☑ Multi-AZ for high availability
- ☑ Monitor everything, alert early
- ☑ Design for failure, not for success ☺

10.4 Scalability Numbers (Approx)

Metric	Value (Approx)
Daily Active Users	10 – 15 Million
Tatkal Peak (10 AM)	~ 2 Million users in few minutes
Requests / Second (Peak)	1 – 2 Lakh RPS
Bookings / Day	~ 10 – 12 Lakh
Databases	Sharded + Read Replicas
Cache Hit Ratio	> 90%
Availability Target	99.95%

10.5 Consistency vs Availability

Strong Consistency (ACID) ⟷ High Availability (Eventual Consistency)

IRCTC chooses a balanced approach

- Critical data (Bookings, Payments) → Strong Consistency
- Non-critical (Reports, Analytics) → Eventual Consistency
- Use Transactions, Locks, Idempotency for correctness.
- Use Caching, Replication for scalability.

10.6 Common Interview Follow-ups

1. How will you handle 10x traffic suddenly?
2. What happens if DB goes down?
3. How do you prevent double booking?
4. What if payment success but booking failed?
5. How do you handle Tatkal queue fairly?
6. How do you ensure data consistency?
7. How will you design PNR generation?
8. How do you handle partial failures?

Think out loud. Break problem, Design step by step. Keep it simple, scalable & reliable! ☺

10.7 Quick Definitions

- PNR : Passenger Name Record – unique booking id.
- Tatkal : Special quota opened 1 day before journey at 10 AM.
- Quota : General, Tatkal, Ladies, Senior Citizen, etc.
- Idempotency : Same request multiple times → same result.
- TTL : Time To Live (for cache keys).
- RPS : Requests Per Second.
- PG : Payment Gateway.

Golden Rules

- ✓ Always design for scale and failure.
- ✓ Keep user experience fast & simple.
- ✓ Data correctness > Everything.
- ✓ Build monitoring, alerts & backups.

Remember

System Design is not about perfect diagrams, it's about problem solving, trade-offs and communication! ☺

All the Best!

Keep learning, keep building and keep shipping!

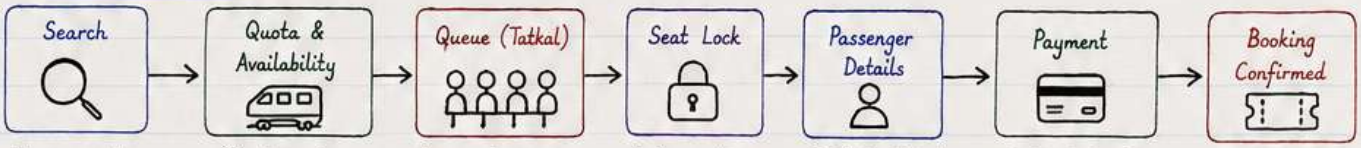
You've Got This! ☺

@codedsoul_05

IRCTC SYSTEM DESIGN
CHEAT SHEET (QUICK REVISION)

CHEAT SHEET 1 PAGE ☆

1. End-to-End Flow (In Short)



User searches trains → Check quota, show availability → User enters queue, gets token → Lock seat in Redis (atomic) → Validate details, apply rules → Redirect to PG, process payment → Generate PNR, update inventory, send ticket

2. Key Components

- Users : Web / App / Agents
- Services : Booking, Inventory, Payment, Notification, Search
- Cache (Redis) : Sessions, Locks, Counters, Rate limiting
- DB (MySQL) : Core data – Users, Trains, Bookings, Payments
- Read Replicas : For read heavy queries
- Queue (Kafka) : Async jobs – emails, sms, notifications
- Storage (S3) : Tickets, Reports, Backups
- Monitoring : Prometheus, Grafana, ELK, Alerts

3. Seat Allocation (Quick)

- Seat states : Available → Locked → Booked
- Use Redis SET per train/date-class
- Seat Lock (NX + Expiry)

```
SET seat_123 lock_token NX EX 120
```

- On payment success → confirm booking & remove lock
- On failure/timeout → release lock
- Waitlist if no seats → auto promote when seat becomes available

4. Payment (Quick)

- Redirect to PG → process payment
- Use order_id as idempotency key
- Payment states :
  INITIATED → PROCESSING → SUCCESS
  FAILED → ALLOW RETRY → REFUNDED
- Use atomic transaction for booking + payment update
- Auto retry for timeout / no response
- Refund via PG / Bank

5. Traffic Handling (Tatkal 10 AM)

1. Virtual Waiting Room
2. FIFO Queue (Fair)
3. Rate Limiting (per IP / device)
4. Token Generation
5. Controlled Batch Entry
6. Exponential Backoff on retries



6. Scaling & Performance

- ✓ Stateless services behind LB
- ✓ Horizontal scaling (auto scale groups)
- ✓ Redis caching (hot data, sessions)
- ✓ Read Replicas for DB reads
- ✓ CDN for static content (images, css, js)
- ✓ Queues for async processing
- ✓ DB Sharding (by date / train / region)
- ✓ Connection pooling & indexing
- ✓ Multi-AZ deployment

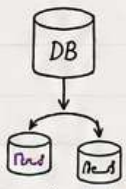
7. Failure Handling

- ☆ Retries with exponential backoff
- ☆ Timeouts & Circuit Breaker
- ☆ Idempotency for all critical APIs
- ☆ Queue for async & at-least-once delivery
- ☆ Graceful degradation (partial results)
- ☆ Auto failover (DB, instances)
- ☆ Regular backups & PITR
- ☆ Monitoring & alerts for quick action

8. Database (Must Know)

Core Tables:
- users, trains, schedule, stations
- quota, seats, bookings, passengers
- payments, refunds, transactions
- waitlist, ncl, trains_logs

Best Practices:
- Proper indexing (date, train_id, station_id)
- Partition by date for big tables
- Use foreign keys & constraints
- Read replicas for scale



9. Important Concepts

- Idempotency Key → Avoid duplicate operations
- Atomicity → Lock, Booking, Payment must be atomic
- TTL → For locks, sessions, tokens
- ACID → Critical for Booking & Payment
- CAP Theorem → Choose availability with consistency where needed
- Eventually Consistent Data → Reports, Analytics
- Monitoring → Measure → Alert → Act

10. Monitoring & Observability

- Metrics : QPS, Latency, Error Rate, CPU, Memory
- Tools : Prometheus, Grafana
- Logs : ELK Stack
- Tracing : Jaeger / Zipkin
- Alerts : PagerDuty / Opsgenie
- DR Plan : RPO, RTO defined & tested



11. Scalability Numbers (Approx)

Metric	Value (Approx)
Daily Active Users	10 – 15 Million
Tatkal Peak (10 AM)	~ 2 Million users
Requests / Second (Peak)	1 – 2 Lakh RPS
Bookings / Day	~ 10 – 12 Lakh
Databases	Sharded + Read Replicas
Cache Hit Ratio	> 90%
Availability Target	99.95%

12. Common Interview Questions

1. How will you handle 10x traffic suddenly?
2. What happens if DB goes down?
3. How do you prevent double booking?
4. What if payment gateway times out?
5. How do you design Tatkal queue fairly?
6. How do you ensure data consistency?
7. How will you design PNR generation?
8. How do you handle partial failures?
9. Which DB indexes will you create & why?
10. How will you monitor the system?



13. Quick Definitions

- PNR : Passenger Name Record
- Tatkal : Quota open 1 day before journey at 10 AM
- Quota : General, Tatkal, Ladies, Senior Citizen, Divyang, etc.
- Idempotency : Same request → same result
- TTL : Time To Live (for keys)
- RPS : Requests Per Second
- PG : Payment Gateway
- CDN : Content Delivery Network

14. Golden Rules

- ☆ Design for scale & failure.
- ☆ Keep user experience fast & simple.
- ☆ Data correctness > Everything.
- ☆ Always monitor, alert & improve.
- ☆ Auto recover, don't rely on manual.



15. Remember

System Design is about trade-offs!
Perfect design doesn't exist.
Keep it simple, scalable, reliable & user friendly. ☺

16. One Line Mantra

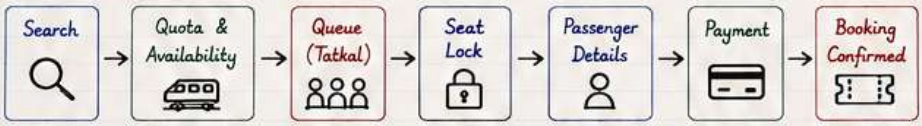
Right Data + Right Time + Right User
= Great System Design

@codedsoul_05

IRCTC SYSTEM DESIGN
CHEAT SHEET (QUICK REVISION)

1 PAGE QUICK REVISION ⭐

1. End-to-End Booking Flow



- Search: User searches trains
- Quota & Availability: Check quota, show availability
- Queue (Tatkal): Enters queue, gets token
- Seat Lock: Lock seat (atomic)
- Passenger Details: Validate details, apply rules
- Payment: Redirect to PG, process payment
- Booking Confirmed: Generate PNR, update inventory, send ticket

2. Key Components

- Users : Web / App / Agents
- Services : Booking, Inventory, Payment, Notification, Search
- Cache (Redis) : Sessions, Locks, Counters, Rate limiting
- DB (MySQL) : Users, Trains, Bookings, Payments, Transactions
- Read Replicas : For read heavy queries
- Queue (Kafka) : Async jobs – emails, sms, notifications
- Storage (S3) : Tickets, Reports, Backups
- Monitoring : Prometheus, Grafana, ELK, Alerts

3. Seat Allocation (Quick)

- Seat States : Available → Locked → Booked
- Use Redis SET per train/date/class
- Seat Lock (NX + Expiry)

```
SET seat_123 lock_token NX EX 120
```

- On payment success → confirm booking & remove lock
- On failure/timeout → release lock
- Waitlist if no seats → auto promote

4. Payment Flow



1. Create order_id & payment session (idempotent)
2. Redirect to PG (Razorpay/PayU/Paytm etc.)
3. Webhook / Return URL → update payment status
4. Success → confirm booking, generate PNR
5. Fail / Timeout → retry / refund (if needed)

Payment States

INITIATED → PROCESSING
SUCCESS → CONFIRMED
FAILED → ALLOW RETRY → REFUNDED (if paid)

5. Traffic Handling (Tatkal 10 AM)

1. Virtual Waiting Room
2. FIFO Queue (Fair)
3. Rate Limiting (per IP/device)
4. Token Generation
5. Controlled Batch Entry
6. Exponential Backoff on retries



6. Scaling & Performance

- ✓ Stateless services behind LB
- ✓ Horizontal scaling (Auto scale groups)
- ✓ Redis caching (hot data, sessions)
- ✓ Read Replicas for DB reads
- ✓ CDN for static content (imags, css, js)
- ✓ Queues for async processing
- ✓ DB Sharding (by date/train/region)
- ✓ Connection pooling & indexing
- ✓ Multi-AZ deployment

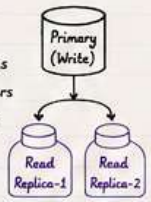
7. Failure Handling (Quick)

- ⭐ Retries with exponential backoff
- ⭐ Timeouts & Circuit Breaker
- ⭐ Idempotency key for all critical APIs
- ⭐ Queue for async & at-least-once delivery
- ⭐ Graceful degradation (partial results)
- ⭐ Auto failover (DB, Instances)
- ⭐ Regular backups & PITR
- ⭐ Monitoring & alerts for quick action

8. Databases (Must Know)

Core Tables :

- users, trains, schedule, stations
- quota, seats, bookings, passengers
- payments, transactions, refunds
- waitlist, audit, trains_logs



Best Practices :

- Proper indexing
- Partition big tables
- Foreign keys & constraints
- Read replicas for scale

9. Monitoring & Observability

- Metrics : QPS, Latency, Error Rate, CPU, Memory
- Tools : Prometheus, Grafana
- Logs : ELK Stack
- Tracing : Jaeger / Zipkin
- Alerts : PagerDuty / Opsgenie
- DR Plan : RPO, RTO defined & tested



10. Important Concepts

- ACID : For critical operations
- Idempotency : Avoid duplicate ops
- TTL : For locks, sessions, tokens
- CAP Theorem : Consistency vs Availability
- Eventual Consistency : For reports, analytics
- Atomicity : Locks, Build, Payment
- CDN : Content Delivery Network

11. Scalability Numbers (Approx)

Metric	Value (Approx)
Daily Active Users	10 – 15 Million
Tatkal Peak (10 AM)	~ 2 Million users in few minutes
Requests / Second (Peak)	1 – 2 Lakh RPS
Bookings / Day	~ 10 – 12 Lakh
Databases	Sharded + Read Replicas
Cache Hit Ratio	> 90%
Availability Target	99.95%

12. One Line Mantra

Right Data + Right Time
+
Right User
=
Great System Design

13. Golden Rules

- ⭐ Design for scale & failure.
- ⭐ Keep user experience fast & simple.
- ⭐ Data correctness > Everything.
- ⭐ Monitor, test & improve continuously.
- ⭐ Auto recover, don't rely on manual.



IRCTC SYSTEM DESIGN – BACK COVER
MOST ASKED INTERVIEW QUESTIONS & ANSWERS

Keep Practicing
You'll Crack It!

1. **How does IRCTC handle 10 AM Tatkal traffic?**
   Ans: Virtual waiting room → FIFO queue → Rate limiting → Token → Controlled batch entry → Spread load on servers.
2. **How is seat locking implemented?**
   Ans: Use Redis SET with NX + Expiry. If lock acquired → seat locked. On success → confirm & remove lock, on failure/timeout → release.
3. **What if two users try to book the same seat?**
   Ans: Redis lock ensures only one gets the lock. Others get failure or waitlisted based on availability.
4. **How is queue implemented?**
   Ans: Using Kafka / Redis Streams. FIFO order maintained, consumers process in order.
5. **How do you ensure payment isn't done twice?**
   Ans: Use idempotency key (order_id) + unique transactions + webhook verification. Duplicate requests return same result.
6. **What if payment is successful but server crashes before booking confirm?**
   Ans: Webhook from PG will update status. On recovery, reconcile pending payments and confirm booking.
7. **How is waitlist managed?**
   Ans: Store waitlist in DB. On seat cancellation, auto promote next waitlist user in order.
8. **What is the purpose of Read Replicas?**
   Ans: Handle read heavy queries (search, availability) without affecting primary DB performance.
9. **How do you achieve high availability?**
   Ans: Multi-AZ deployment, auto scaling, DB replication, health checks, failover, load balancer.
10. **How do you handle database failure?**
    Ans: Primary failover to replica using automated failover. Regular backups and PITR for recovery.
11. **How do you prevent bots and unfair usage?**
    Ans: CAPTCHA, rate limiting, device fingerprinting, account limits, WAF.
12. **Why use CDN?**
    Ans: Serve static content (images, css, js) from edge locations → reduces latency & load on origin servers.
13. **How do you store session and locks?**
    Ans: Redis (in-memory) for fast access, TTL for auto expiry.
14. **Why is idempotency important?**
    Ans: Network retries can cause duplicate requests. Idempotency ensures same request doesn't create multiple bookings/payments.
15. **How do you handle reports & analytics?**
    Ans: Use separate reporting DB / Data warehouse. ETL or CDC to sync data from primary.
16. **What is eventual consistency?**
    Ans: Data may not be immediately consistent across all systems (e.g., reports). It becomes consistent over time.
17. **How do you design for failure?**
    Ans: Retries, timeouts, circuit breaker, graceful degradation, monitoring, alerts, auto recovery.
18. **What is TTL used for?**
    Ans: For cache keys, locks, sessions, tokens – auto expiry to free resources.
19. **How do you scale databases?**
    Ans: Sharding (by date/train/region), read replicas, partitioning, indexing.
20. **What metrics will you monitor?**
    Ans: QPS, Latency, Error Rate, CPU, Memory, DB Connections, Cache Hit Ratio, Payment Success Rate.

⭐ Focus on Fundamentals, Think Like a System Designer, Practice Consistently and You'll Succeed!